OCapN Promise Shortening

Ridley

Acknowledgement

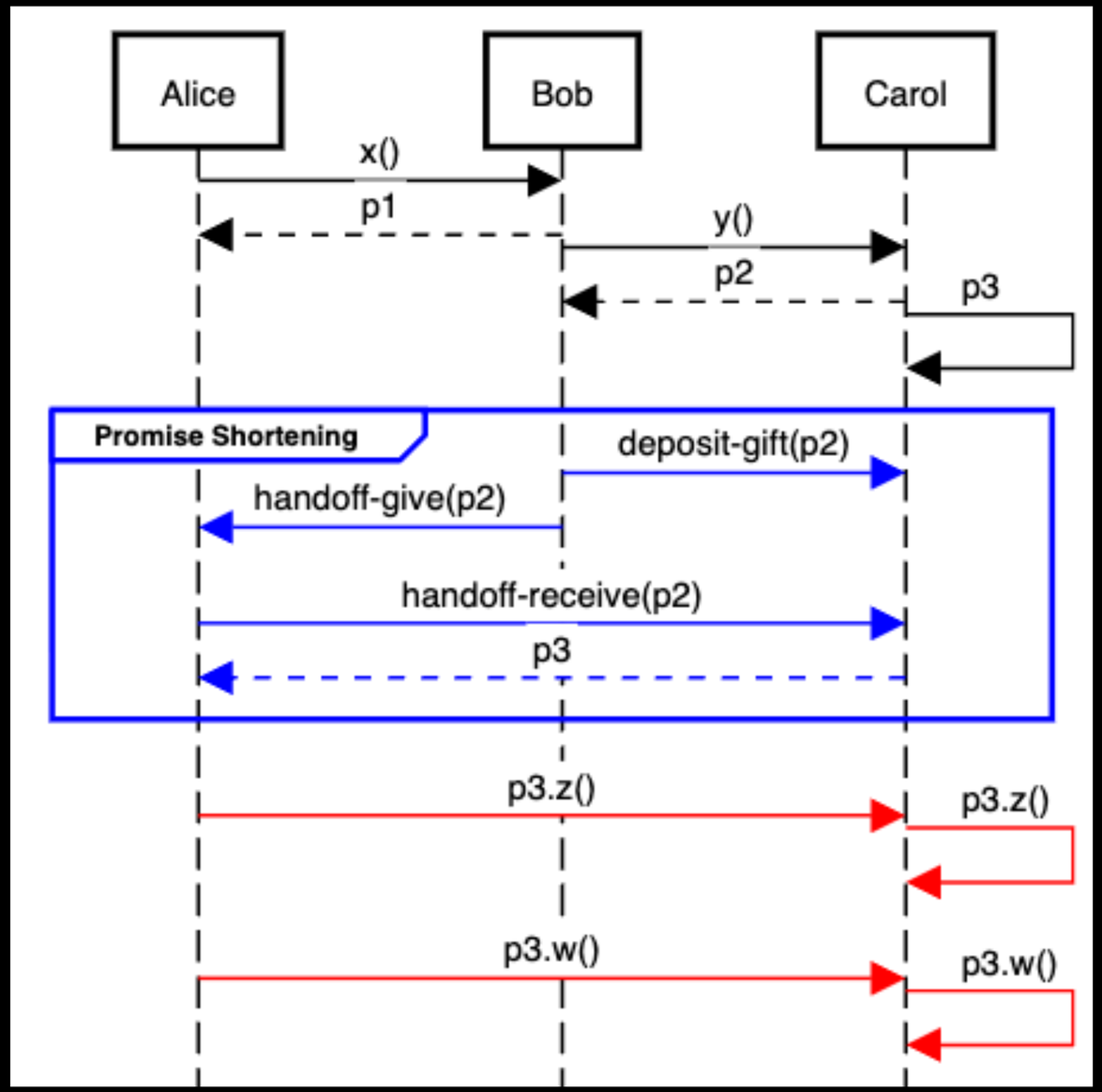
- Thanks to Mark Miller for providing the example that highlighted the issue
- This proposal is largely inspired by how I understand this problem was solved in E

Our Example Scenario

- Three peers: Alice, Bob, Carol
- Alice calls x() on Bob (returning p1), which calls y() on Carol (returning p2)
- Alice calls p1.z()
- Alice calls p1.w()
- Alice’s expectation: The settled result of p1 receives z() before w()

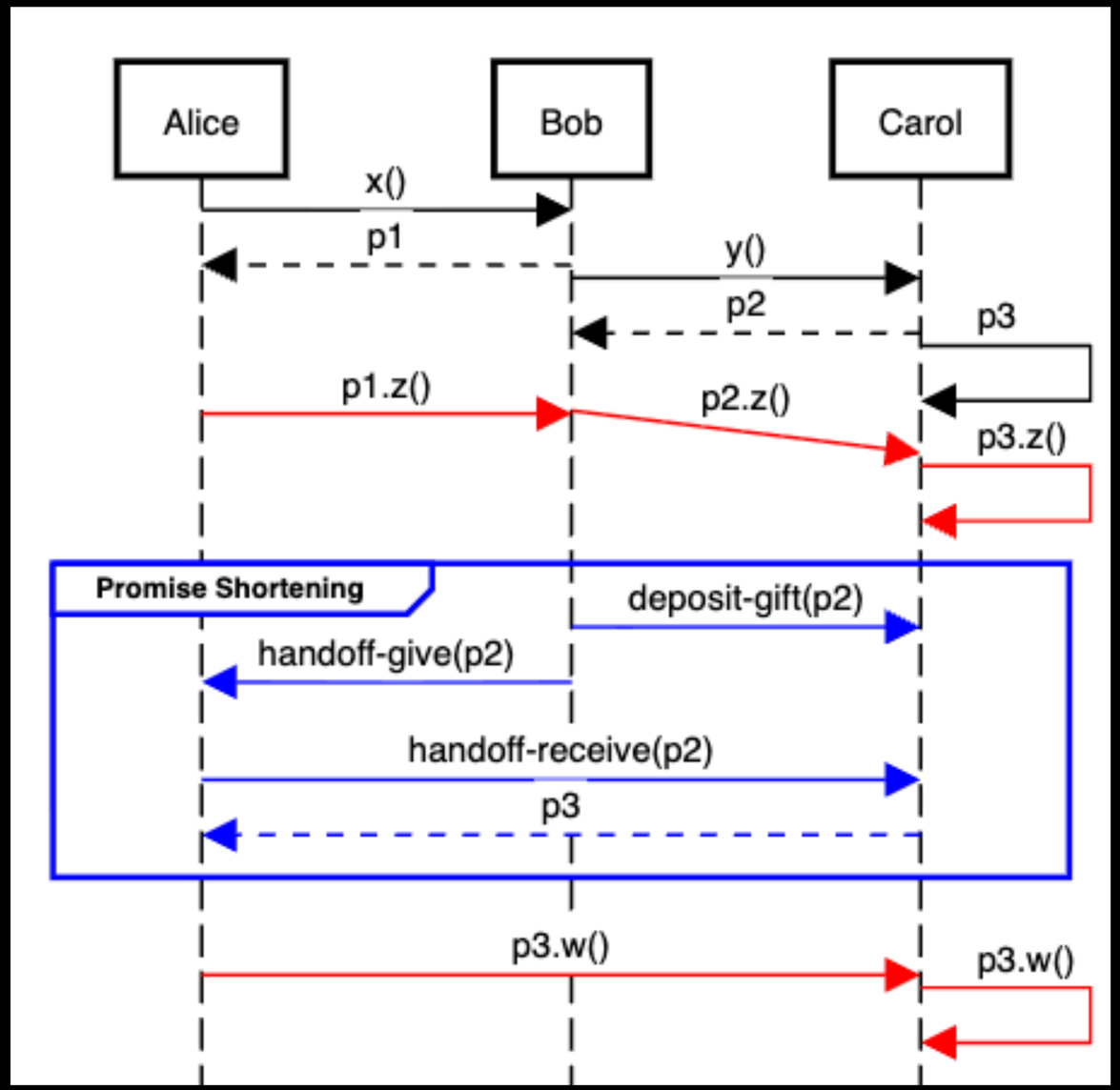
Existing Happy Scenario

- Alice calls x() on Bob, which calls y() on Carol
- Promise shortening occurs
- Alice calls p1.z()
- Alice calls p1.w()
- p3.z() happens before p3.w() as Alice expects!



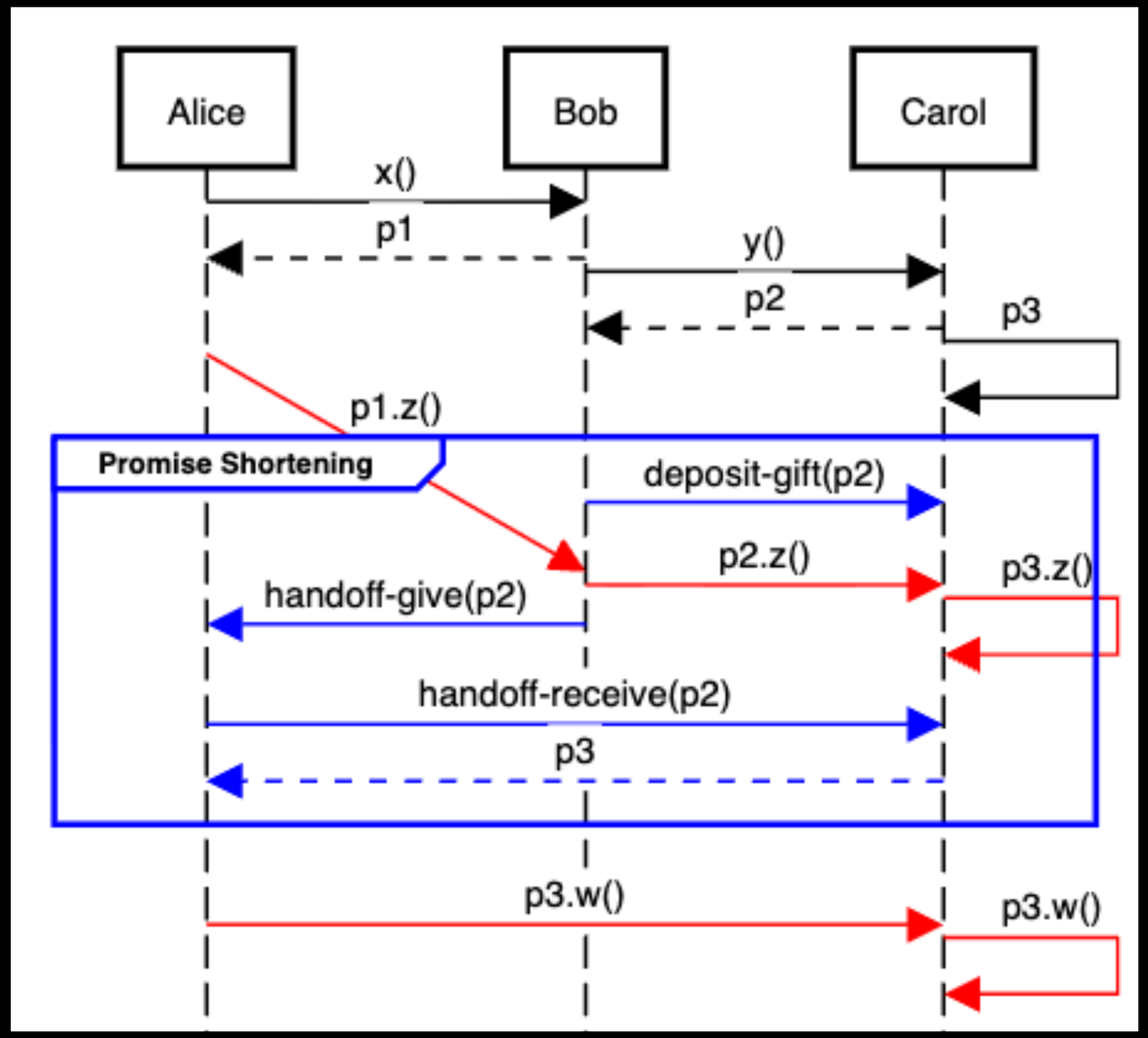
Existing Lucky Scenario 1

- Alice calls x() on Bob, which calls y() on Carol
- Alice calls p1.z()
- Promise shortening occurs
- Alice calls p1.w()
- p3.z() happens before p3.w() as Alice expects!



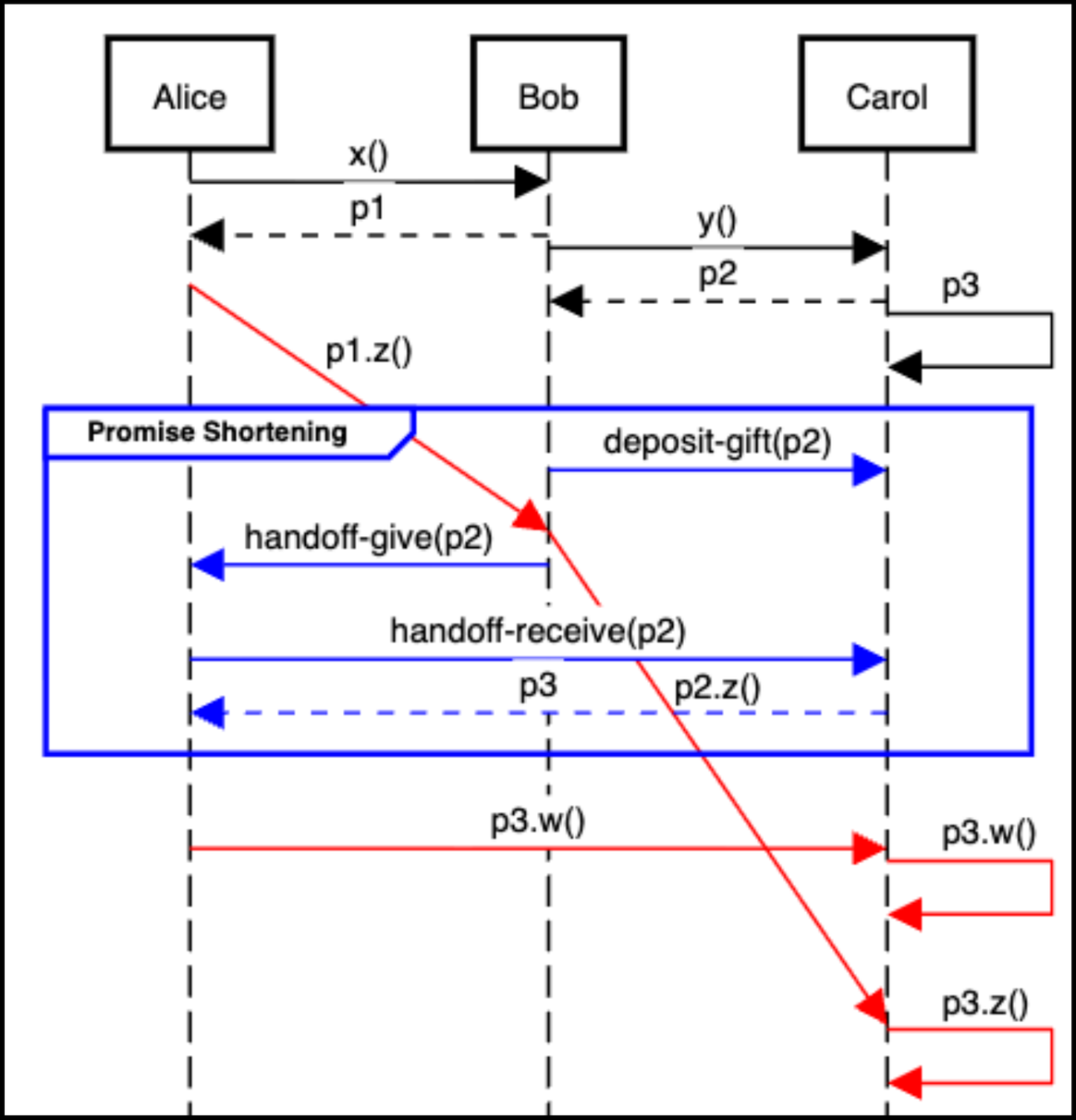
Existing Lucky Scenario 2

- Alice calls x() on Bob, which calls y() on Carol
- Alice calls p1.z()
- Promise shortening occurs
- Alice calls p1.w()
- p3.z() happens before p3.w() as Alice expects!



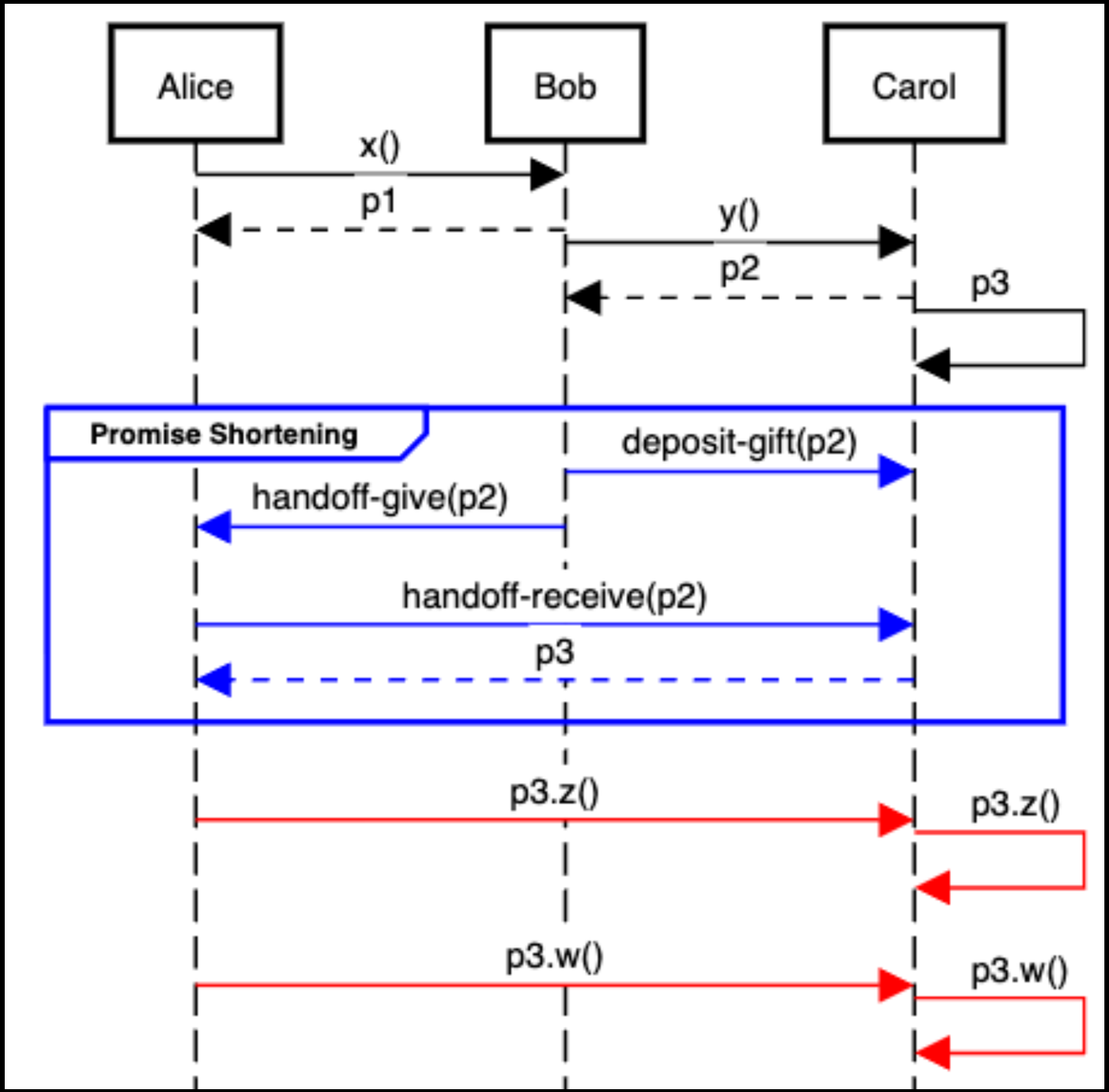
Existing Nightmare Scenario

- Alice calls x() on Bob, which calls y() on Carol
- Alice calls p1.z()
- Promise shortening occurs
- Alice calls p1.w()
- p3.z() happens AFTER p3.w()!!! Alice does not expect this!



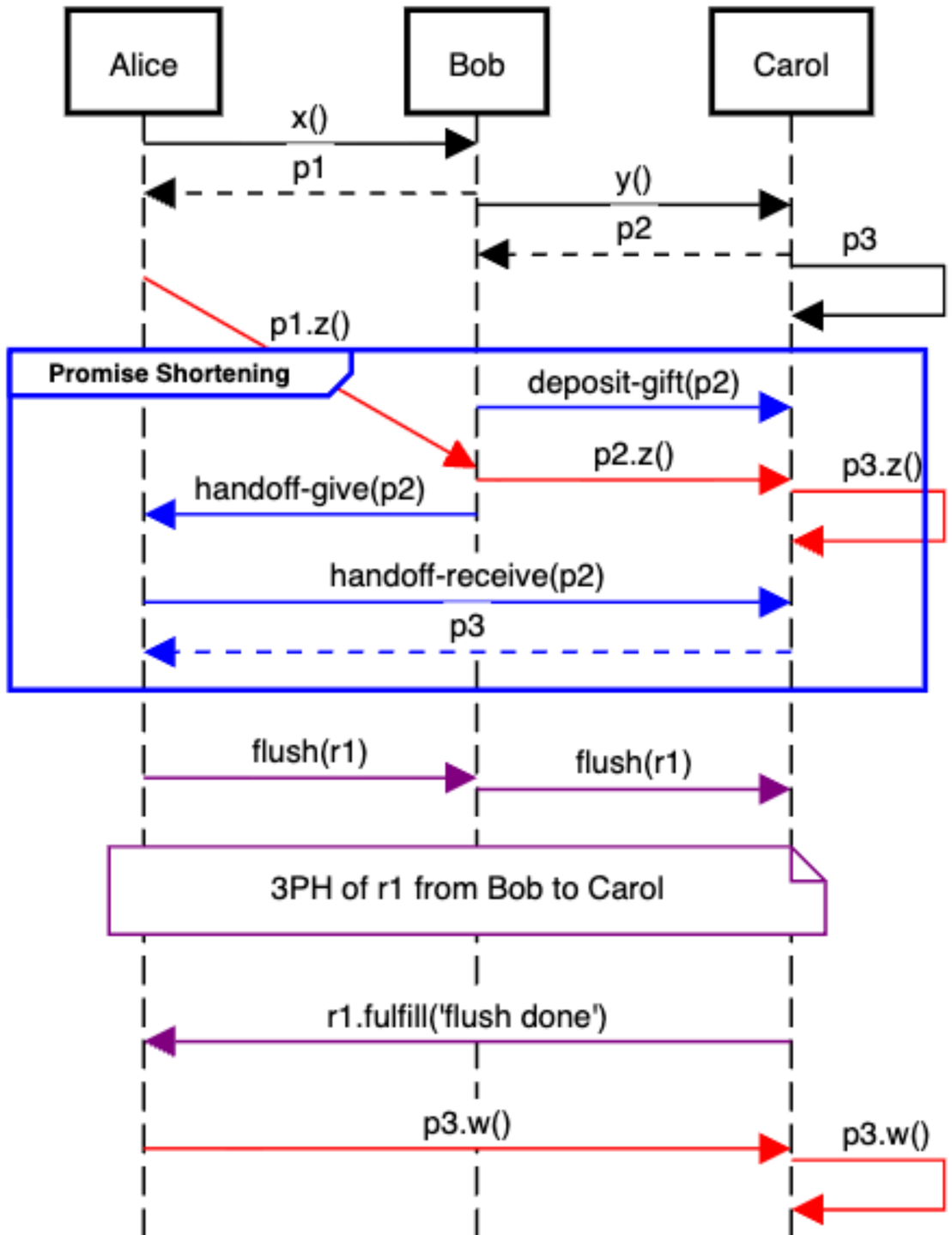
Proposal in Happy Scenario

- No change from the existing happy scenario!



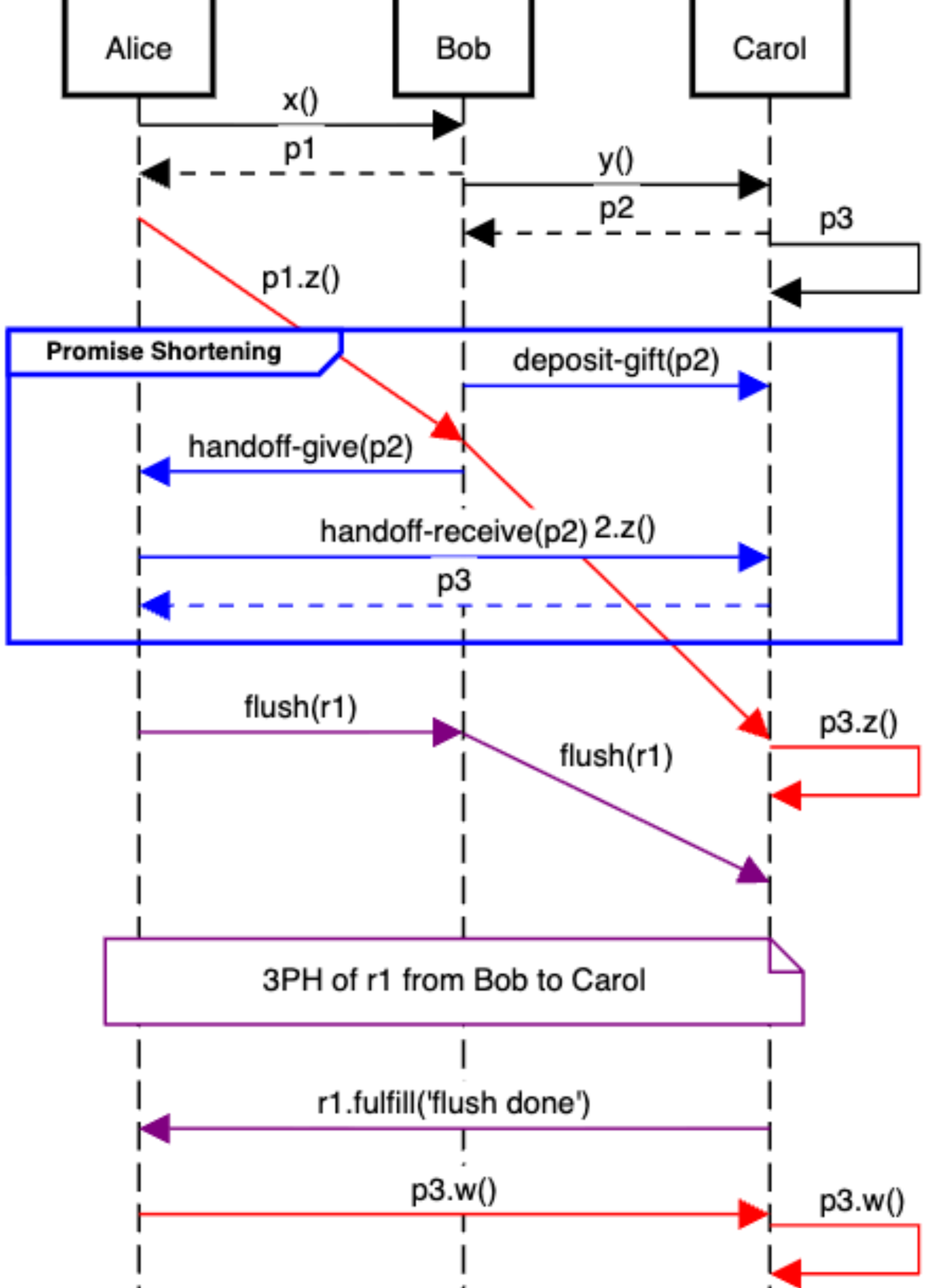
Proposal in Lucky Scenario 2

- Alice calls x() on Bob, which calls y() on Carol
- Alice calls p1.z()
- Alice calls p1.w()
- After promise shortening, flushing is performed before Alice's w() call is sent!
- p3.z() happens before p3.w() as Alice expects!

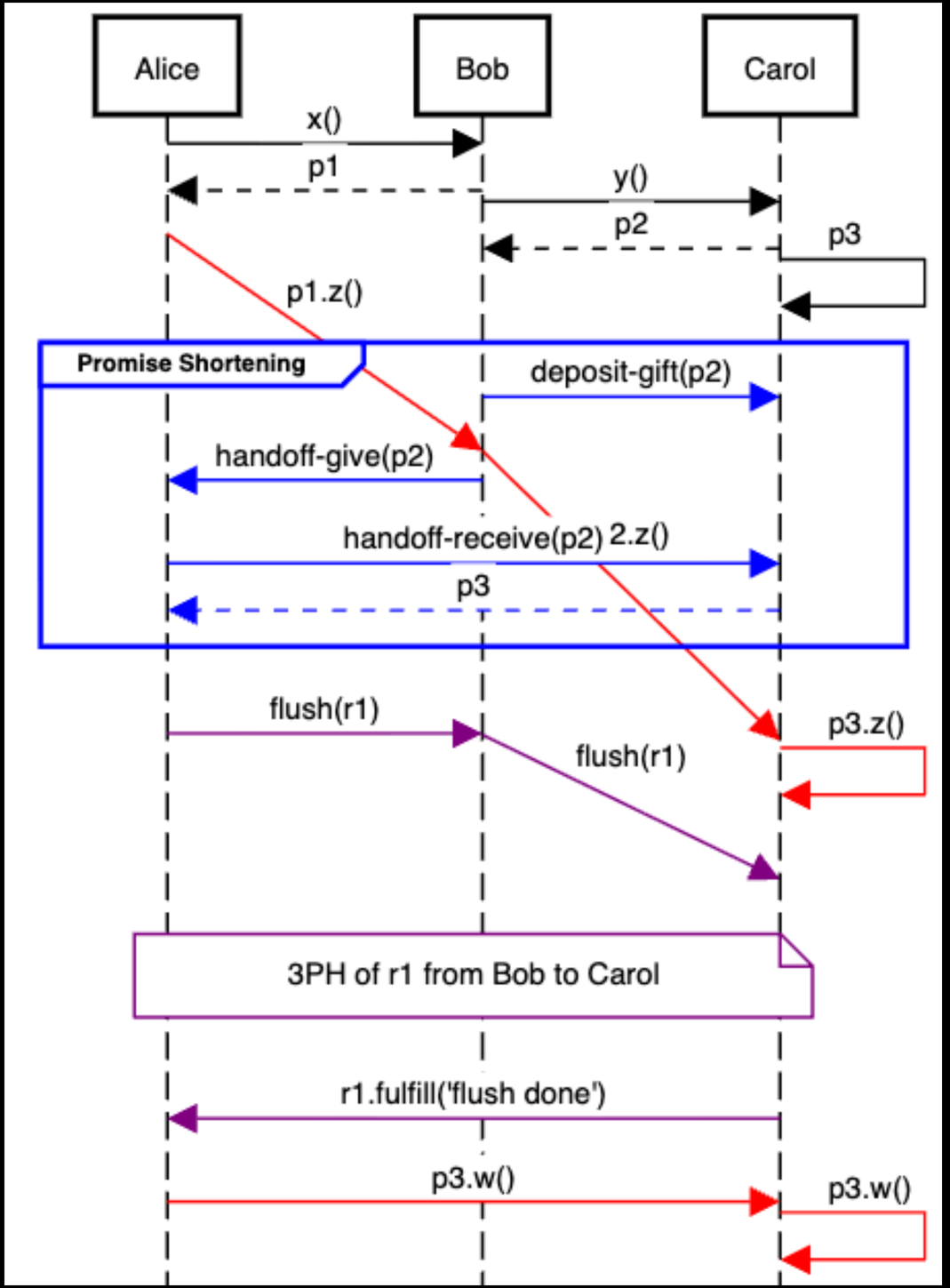


Proposal in Nightmare Scenario

- Alice calls x() on Bob, which calls y() on Carol
- Alice calls p1.z()
- Alice calls p1.w()
- After promise shortening, p3.w() is buffered until flushing is confirmed!
- p3.z() happens before p3.w() just as Alice expects!



Points of Discussion

Assuming people are on board so far

- Current proposal adds only one OCapN op (op:flush) used in two modes. Some might prefer two ops.
- Scenarios in which r1 is broken
- Could use an op instead of r1 but means more complicated implementation
- Replace “flush done” with something shorter